

Introduction to Derivatives (Optional)

This optional section introduces the mathematical concept of a **derivative**, which is the core component of **backpropagation**. Backpropagation is the algorithm that TensorFlow uses to automatically calculate the derivatives needed for gradient descent (or the Adam optimizer).

This lesson uses an intuitive, non-calculus-based approach to understand what a derivative is.

An Intuitive Definition of a Derivative

A derivative measures the **sensitivity** or **effect** that a tiny change in one variable has on another.

For a cost function $J(w)$, the derivative of J with respect to w (written as $\frac{\partial J}{\partial w}$) answers the question: "If I nudge w by a tiny amount, how much does J change in response?"

The "Epsilon Nudge" Method

Let's use a simple cost function: $J(w) = w^2$

1. **Start with a value:** Let $w = 3$.
 - The cost $J(3) = 3^2 = 9$.
 2. **Nudge the variable:** Increase w by a tiny amount, ϵ (epsilon). Let's say $\epsilon = 0.001$.
 - Our new w is $3 + 0.001 = 3.001$.
 3. **Calculate the new cost:**
 - $J(3.001) = (3.001)^2 = 9.006001$.
 4. **Analyze the change:**
 - We increased w by 0.001 (ϵ).
 - The cost J increased by 0.006001.
 - This change in J (0.006001) is almost exactly **6** times the change in w (0.001).
 5. **Conclusion:** The **derivative is 6**.
 - **Informal Definition:** If w goes up by ϵ , and $J(w)$ goes up by $k \times \epsilon$, then the derivative is k .
-

How Derivatives Power Gradient Descent

The gradient descent update rule is: $w := w - \alpha \frac{\partial J}{\partial w}$

The derivative ($\frac{\partial J}{\partial w}$) tells the algorithm how big of a step to take.

- **Large Derivative:** If the derivative is large (like 6), it means changing w has a **big effect** on the cost J . Therefore, gradient descent should take a **large step** to get a big improvement.
- **Small Derivative:** If the derivative is small, changing w has a **small effect** on J . Therefore, gradient descent should take a **small step**.

Key Properties of Derivatives

1. The Derivative Depends on the Value of w

The derivative is not a fixed number; it changes depending on the *current value* of w . For our function $J(w) = w^2$:

- If $w = 3$, $\frac{\partial J}{\partial w} = 6$.
- If $w = 2$, $\frac{\partial J}{\partial w} = 4$. (A 0.001 nudge makes J go from 4 to ~ 4.004).
- If $w = -3$, $\frac{\partial J}{\partial w} = -6$. (A +0.001 nudge to w makes J decrease by 0.006, so the change is $-6 \times \epsilon$).

2. The Calculus Connection (Optional)

- **Slope:** In calculus, the derivative is formally defined as the **slope of the line tangent** to the function at that exact point.
- **Formula:** For $J(w) = w^2$, the derivative formula is $\frac{\partial J}{\partial w} = 2w$.
- This formula perfectly matches our intuitive findings:
 - At $w = 3$, derivative = $2 \times 3 = 6$.
 - At $w = 2$, derivative = $2 \times 2 = 4$.
 - At $w = -3$, derivative = $2 \times (-3) = -6$.

3. Examples of Other Derivatives

The video uses the Python library `SymPy` to find the derivative formulas for other functions:

Function ()	Derivative ()	Value at
w^2	$2w$	4
w^3	$3w^2$	12
w	1	1

Function ($J(w)$)	Derivative ($\frac{\partial J}{\partial w}$)	Value at $w = 2$
$1/w$	$-1/w^2$	-0.25

4. A Note on Notation

- You may see two symbols for derivatives: d/dw (total derivative) and $\partial/\partial w$ (partial derivative).
- The "squiggly d" (∂) is used when a function (like our cost J) depends on *more than one variable* (e.g., $w_1, w_2, \dots, b_1, b_2, \dots$).
- Since our cost function *always* depends on many parameters, this course will **always use the partial derivative symbol $\partial/\partial w$** .

The Computation Graph (Optional)

The **computation graph** is a key idea in deep learning. It's how frameworks like TensorFlow visualize a complex function and efficiently compute derivatives. It breaks down a large calculation into a sequence of small, simple steps.

1. Forward Propagation (Left-to-Right) ➡

Forward propagation (or "forward prop") is the process of calculating the final output (the cost J) by moving from left to right through the graph.

Let's use a simple linear regression model as an example:

- **Model:** $a = wx + b$
- **Cost:** $J = \frac{1}{2}(a - y)^2$
- **Values:** $w = 2, b = 8, x = -2, y = 2$

We can break this down into a graph:

1. **Node 1:** $c = w \times x$
 - $c = 2 \times (-2) = -4$
2. **Node 2:** $a = c + b$
 - $a = (-4) + 8 = 4$
3. **Node 3:** $d = a - y$
 - $d = 4 - 2 = 2$
4. **Node 4 (Final Cost):** $J = \frac{1}{2}d^2$
 - $J = \frac{1}{2}(2)^2 = 2$

2. Backpropagation (Right-to-Left) ⬅

Backpropagation (or "backprop") is the process of computing the derivatives by moving backward (right-to-left) through the graph. It uses the derivative's "nudging" concept to see how each node in the graph affects the final cost J .

We want to find $\frac{\partial J}{\partial w}$ and $\frac{\partial J}{\partial b}$.

1. **Start at the end (Node 4 → Node 3):** Find $\frac{\partial J}{\partial d}$
 - $J = \frac{1}{2}d^2$. We have $d = 2$.
 - Using the calculus rule for d^2 , the derivative is just d .
 - So, $\frac{\partial J}{\partial d} = d = 2$.
 - *Intuition:* If d is nudged by ϵ , J changes by $2 \times \epsilon$.

2. **Move to Node 2:** Find $\frac{\partial J}{\partial a}$

- We know $d = a - y$. If we nudge a by ϵ , d also changes by ϵ .
- Since d changing by ϵ causes J to change by $2 \times \epsilon$ (from step 1), a nudge in a also causes J to change by $2 \times \epsilon$.
- Therefore, $\frac{\partial J}{\partial a} = 2$.

3. **Move to Node 1:** Find $\frac{\partial J}{\partial c}$ and $\frac{\partial J}{\partial b}$

- We know $a = c + b$.
- If we nudge c by ϵ , a also changes by ϵ . This causes J to change by $2 \times \epsilon$.
- Therefore, $\frac{\partial J}{\partial c} = 2$.
- If we nudge b by ϵ , a also changes by ϵ . This causes J to change by $2 \times \epsilon$.
- Therefore, $\frac{\partial J}{\partial b} = 2$. **We found one of our derivatives!**

4. **Move to the start:** Find $\frac{\partial J}{\partial w}$

- We know $c = w \times x$, and $x = -2$. So, $c = -2w$.
- If we nudge w by ϵ , c changes by $-2 \times \epsilon$.
- From step 3, we know that any change in c causes a $2 \times$ change in J .
- So, a nudge of ϵ in w causes a change of $(-2 \times \epsilon) \times 2 = -4 \times \epsilon$ in J .
- Therefore, $\frac{\partial J}{\partial w} = -4$. **We found our other derivative!**

Why Backprop is Efficient

- **Reusability:** Backprop is efficient because it **reuses calculations**. Notice that to find the derivatives for *both* w and b , we needed to know $\frac{\partial J}{\partial a}$. We only had to compute this value *once* and then reused it for both backward paths.
- In a large network, this reusability saves an enormous amount of computation, making it far more efficient than calculating each derivative independently from scratch.

Backpropagation Example on a Neural Network (Optional)

This lesson walks through a concrete example of forward and backward propagation on a simple neural network to build intuition for how the backpropagation algorithm works and why it is so efficient.

The Example Network & Setup

- **Architecture:** A simple 2-layer network.
 - 1 hidden layer with 1 neuron (a_1).
 - 1 output layer with 1 neuron (a_2).
 - **Activation:** ReLU ($g(z) = \max(0, z)$) for all layers.
 - **Cost Function:** Squared Error, $J = \frac{1}{2}(a_2 - y)^2$.
 - **Training Example:** $x = 1, y = 5$.
 - **Parameters:**
 - Layer 1: $w_1 = 2, b_1 = 0$
 - Layer 2: $w_2 = 3, b_2 = 1$
-

1. Forward Propagation (Left-to-Right →)

We first calculate the output and the final cost by moving forward through the network.

1. Layer 1:

- $z_1 = w_1x + b_1 = (2 \times 1) + 0 = 2$
- $a_1 = g(z_1) = \max(0, 2) = 2$

2. Layer 2:

- $z_2 = w_2a_1 + b_2 = (3 \times 2) + 1 = 7$
- $a_2 = g(z_2) = \max(0, 7) = 7$

3. Cost:

- $J = \frac{1}{2}(a_2 - y)^2 = \frac{1}{2}(7 - 5)^2 = \frac{1}{2}(2)^2 = 2$

This entire process can be mapped to a **computation graph** showing each step.

2. Backpropagation (Right-to-Left ←)

Backpropagation is the process of moving backward through the graph (from right to left) to calculate the derivative of the final cost J with respect to every parameter.

Without showing every intermediate step (which was covered in the previous video), this process would find the derivatives for all four parameters:

- $\frac{\partial J}{\partial w_1} = 6$
- $\frac{\partial J}{\partial b_1} = 2$
- $\frac{\partial J}{\partial w_2} = 2$
- $\frac{\partial J}{\partial b_2} = 2$

The key result we will verify is $\frac{\partial J}{\partial w_1} = 6$.

3. Verifying the Derivative (The "Nudge" Check) 🧑🏻‍🔬

We can confirm the result of backprop by manually "nudging" a parameter and re-running forward propagation to see the effect on the final cost J .

- **Goal:** Check if $\frac{\partial J}{\partial w_1} = 6$ is correct.
 - **Original values:** $w_1 = 2$, $J = 2$.
 - **Nudge w_1 :** Let's increase w_1 by a tiny amount, $\epsilon = 0.001$.
 - $w_1^{\text{new}} = 2.001$
 - **Recalculate Forward Prop:**
 1. **Layer 1:**
 - $z_1 = (2.001 \times 1) + 0 = 2.001$
 - $a_1 = \max(0, 2.001) = 2.001$
 2. **Layer 2:**
 - $z_2 = (3 \times 2.001) + 1 = 6.003 + 1 = 7.003$
 - $a_2 = \max(0, 7.003) = 7.003$
 3. **Cost:**
 - $J^{\text{new}} = \frac{1}{2}(7.003 - 5)^2 = \frac{1}{2}(2.003)^2 \approx 2.006005$
 - **Analyze the Change:**
 - We increased w_1 by 0.001 (ϵ).
 - The cost J increased from 2.000 to ≈ 2.006 .
 - The change in J is ≈ 0.006 , which is **6 times** the change in w_1 .
 - **Conclusion:** This confirms that the derivative $\frac{\partial J}{\partial w_1}$ is indeed **6**.
-

Why Backpropagation is Efficient ⚡

- **The "Nudge" Method (Numerical Differentiation):** The check we just performed is a valid way to find a derivative. However, to find all 4 derivatives, we would have to run this **entire forward propagation process 4 times** (once for each parameter).
 - For P parameters and N nodes, this takes $O(N \times P)$ steps. This is **very slow** for large networks.
- **Backpropagation (Automatic Differentiation):**
 - Backprop is a clever algorithm that moves right-to-left, **reusing calculations** from later layers to compute derivatives for earlier layers.
 - It can compute *all* derivatives in roughly $O(N + P)$ steps, which is **vastly more efficient**.
 - This is the "magic" that frameworks like TensorFlow use. The process is called **automatic differentiation (autodiff)**.